

CO3 – Assessment Tool 1: Experiments

Q1. Distribution Diagnosis using ECDF and Q-Q Plot

Given data

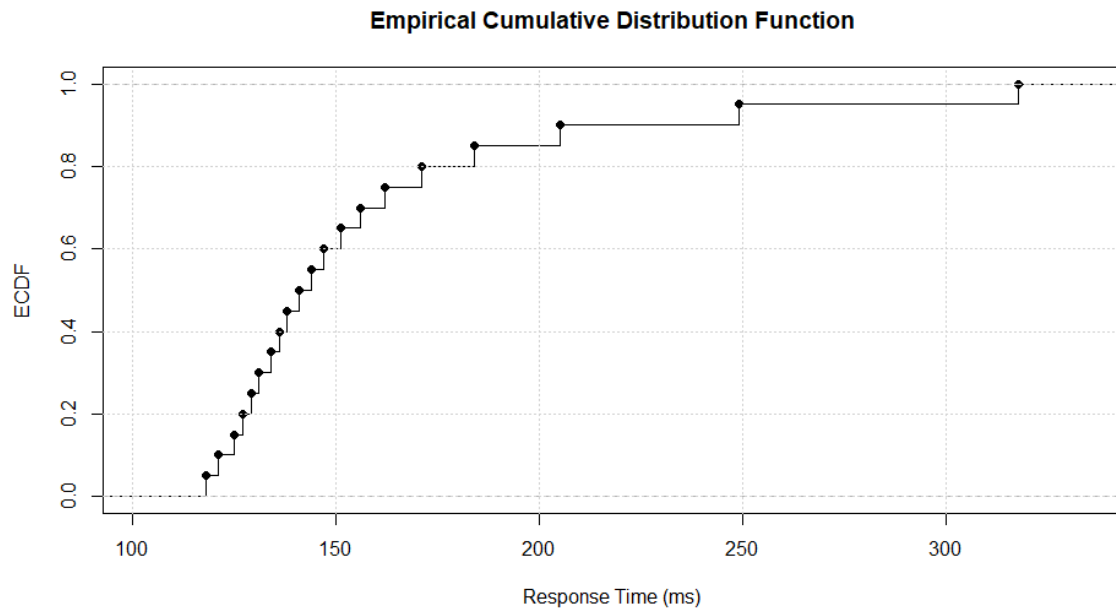
```
response_time <- c(
  118, 121, 125, 127, 129, 131, 134, 136, 138, 141,
  144, 147, 151, 156, 162, 171, 184, 205, 249, 318
)
```

R Code – ECDF

```
response_time <- c(
  118, 121, 125, 127, 129, 131, 134, 136, 138, 141,
  144, 147, 151, 156, 162, 171, 184, 205, 249, 318
)

# ECDF
plot(
  ecdf(response_time),
  main = "Empirical Cumulative Distribution Function",
  xlab = "Response Time (ms)",
  ylab = "ECDF",
  verticals = TRUE,
  do.points = TRUE
)

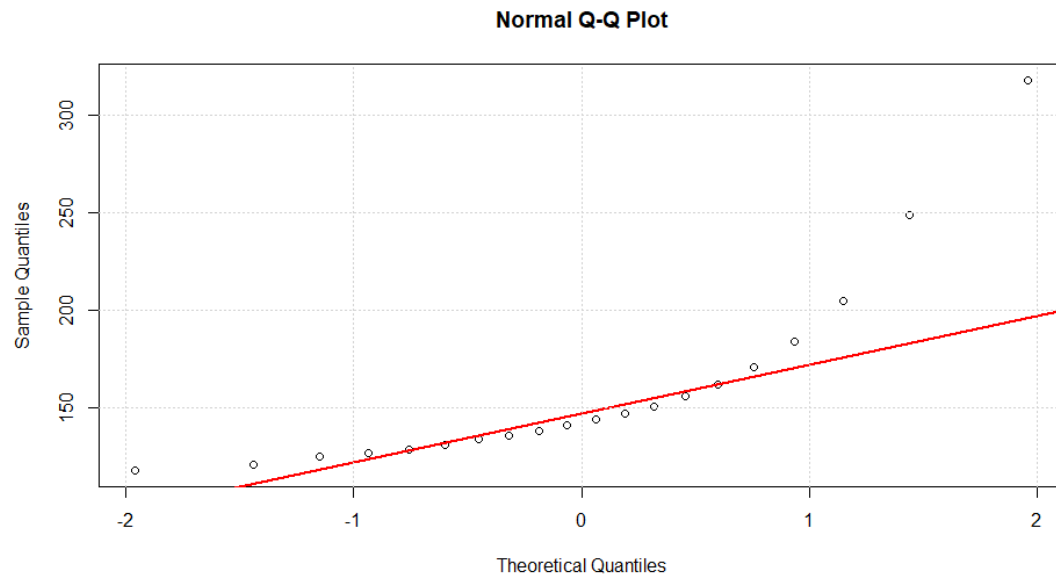
grid()
```



R Code – Normal Q-Q Plot

```
qqnorm(  
  response_time,  
  main = "Normal Q-Q Plot"  
)
```

```
qqline(  
  response_time,  
  col = "red",  
  lwd = 2  
)
```



grid()

Calculate Mean, Median and Standard Deviation

```
mean(response_time)
```

```
median(response_time)
```

```
sd(response_time)
```

Results

```
[  
Mean = 159.35\text{ ms}  
]
```

```
[  
Median = 142.5\text{ ms}  
]
```

```
[  
SD \approx 49.00\text{ ms}  
]
```

Since:

```
[  
Mean > Median  
]
```

the distribution is **positively/right skewed**.

Interpretation

The ECDF shows that most response times are concentrated in the lower range, while a few observations extend far toward higher values.

The observations:

205, 249, 318

form a long upper tail.

The Q-Q plot will show deviations from the straight reference line, especially in the upper tail. Therefore, the data are **not well described by a normal distribution**.

Final answer

The response-time data are positively skewed and contain extreme upper-tail observations. The Q-Q plot deviates from the normal reference line, while the ECDF shows a long upper tail. Therefore, the normality assumption is not strongly supported. Percentile-based monitoring limits are preferable to mean $\pm$ standard deviation limits.

Monitoring limits

```
mean_rt <- mean(response_time)
```

```
sd_rt <- sd(response_time)
```

```
lower_limit <- mean_rt - sd_rt
```

```
upper_limit <- mean_rt + sd_rt
```

```
lower_limit
```

```
upper_limit
```

Approximately:

```
[  
110.35\text{ ms to }208.35\text{ ms}  
]
```

However, because the data are skewed, **percentile-based limits are recommended**.

For example:

```
quantile(response_time, c(0.05, 0.95))
```

Q2. Comparing Four Distributions

Given data

```
L1 <- c(42, 44, 45, 46, 47, 48, 49, 50, 51, 53)
```

```
L2 <- c(38, 41, 44, 47, 50, 53, 56, 59, 62, 65)
```

```
L3 <- c(45, 46, 46, 47, 47, 48, 48, 49, 49, 50)
```

```
L4 <- c(40, 41, 42, 43, 44, 45, 48, 55, 61, 72)
```

```
cycle_data <- list(L1, L2, L3, L4)
```

1. Box Plot

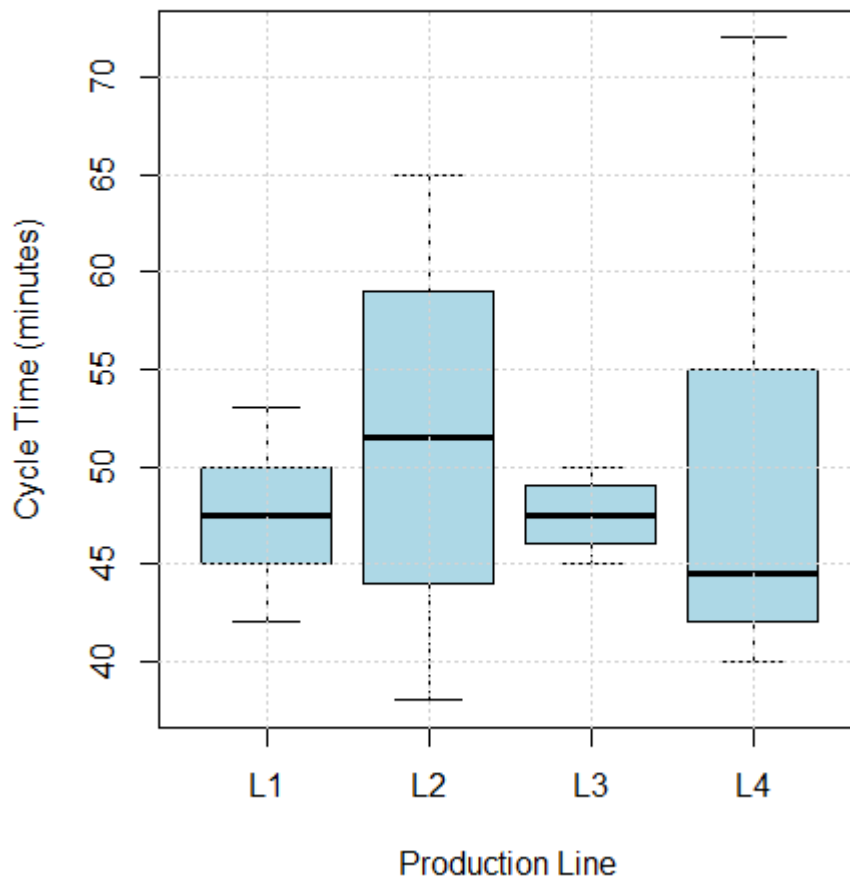
```
boxplot(  
  cycle_data,  
  names = c("L1", "L2", "L3", "L4"),  
  main = "Cycle-Time Distribution by Production Line",  
  xlab = "Production Line",  
  ylab = "Cycle Time (minutes)",  
  col = "lightblue"  
)
```

```
grid()
```

This allows us to compare:

- Median
- Spread
- Quartiles
- Outliers
- Distribution variability

Cycle-Time Distribution by Production Line



2. Violin Plot – Bean Plot Equivalent

A violin plot can be used as an equivalent distribution visualization.

First install the package if required:

```
install.packages("ggplot2")
```

Then:

```
library(ggplot2)
```

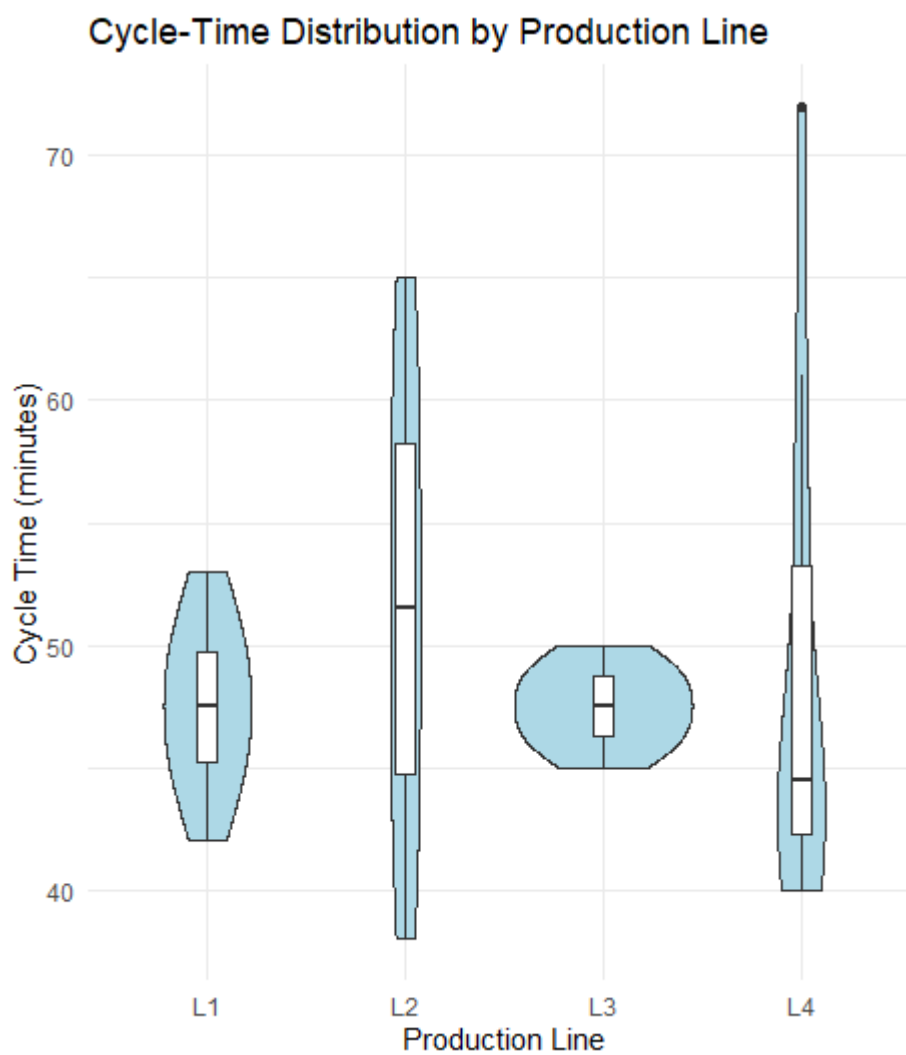
```
df <- data.frame(
```

```
  Line = rep(c("L1", "L2", "L3", "L4"), each = 10),
```

```
  CycleTime = c(L1, L2, L3, L4)
```

)

```
ggplot(df, aes(x = Line, y = CycleTime)) +  
  geom_violin(fill = "lightblue") +  
  geom_boxplot(width = 0.1, fill = "white") +  
  labs(  
    title = "Cycle-Time Distribution by Production Line",  
    x = "Production Line",  
    y = "Cycle Time (minutes)"  
  ) +  
  theme_minimal()
```



3. Calculate Mean and Standard Deviation

mean(L1)

sd(L1)

mean(L2)

sd(L2)

mean(L3)

sd(L3)

mean(L4)

sd(L4)

Results

Line	Mean	SD	Range
------	------	----	-------

L1	47.5	3.37	11
----	------	------	----

L2	51.5	9.08	27
----	------	------	----

L3	47.5	1.58	5
----	------	------	---

L4	49.1	10.44	32
----	------	-------	----

Greatest variability

L4 has the greatest variability:

```
[  
SD_{L4}=10.44  
]
```

and:

```
[  
Range=72-40=32  
]
```

Therefore:

L4 has the greatest variability.

Most concentrated

L3 has:

[
 $SD_{\{L3\}}=1.58$
]

and:

[
 $\text{Range}=50-45=5$
]

Therefore:

L3 is the most concentrated/consistent distribution.

Important interpretation

L1 and L3 both have the same mean:

[
47.5
]

But their standard deviations are different:

[
 $SD_{\{L1\}}=3.37$
]

[
 $SD_{\{L3\}}=1.58$
]

Therefore, comparing only means is insufficient.

Final answer

L4 has the greatest variability because it has the highest standard deviation and range.

L3 is the most concentrated because its observations have the smallest standard deviation and range. Distribution plots are necessary because means alone cannot reveal spread, skewness, or extreme observations.

Q3. Proportion Visualization

Given data

Department Online Blended Face-to-Face

CSE	52%	28%	20%
ECE	45%	35%	20%
IT	30%	40%	30%

1. Side-by-Side Bar Chart

```
departments <- c("CSE", "ECE", "IT")
```

```
online <- c(52, 45, 30)
```

```
blended <- c(28, 35, 40)
```

```
face_to_face <- c(20, 20, 30)
```

```
data_matrix <- rbind(  
  Online = online,  
  Blended = blended,  
  Face_to_Face = face_to_face  
)
```

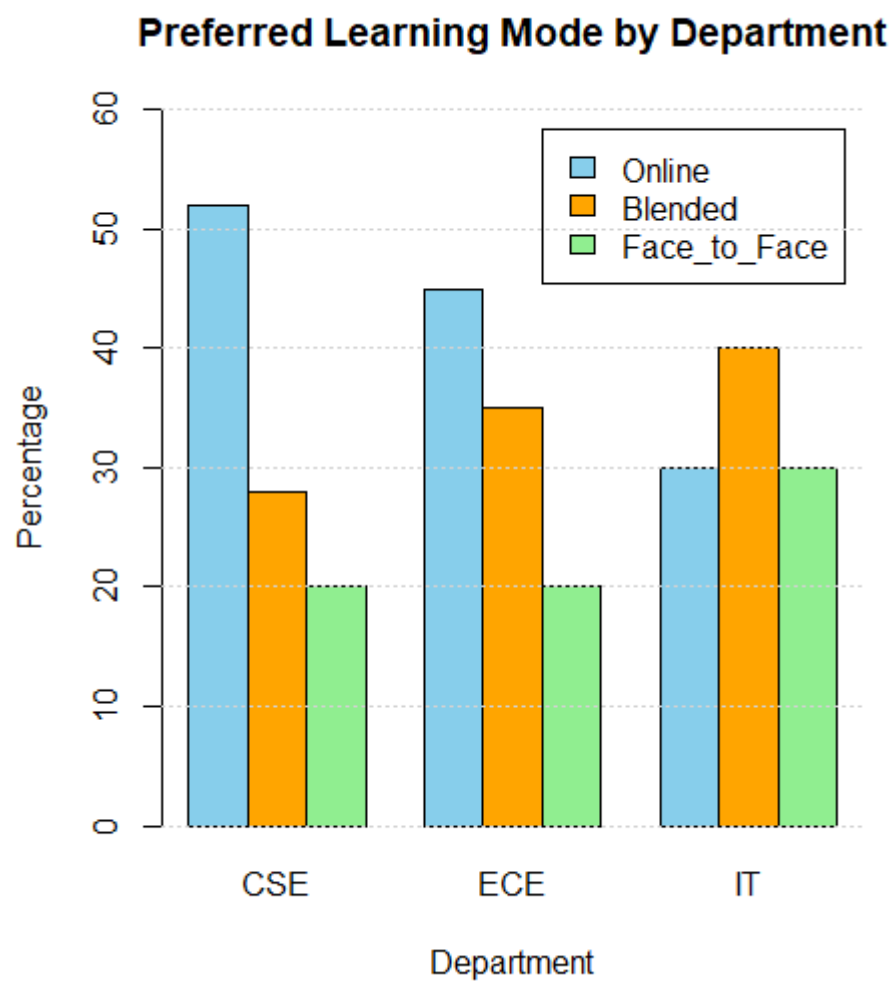
```
barplot(  
  data_matrix,  
  beside = TRUE,  
  names.arg = departments,  
  col = c("skyblue", "orange", "lightgreen"),  
  main = "Preferred Learning Mode by Department",  
  xlab = "Department",
```

```

ylab = "Percentage",
ylim = c(0, 60),
legend.text = TRUE
)

grid(nx = NA, ny = NULL)

```



2. 100% Stacked Bar Chart

```

barplot(
  data_matrix,
  beside = FALSE,

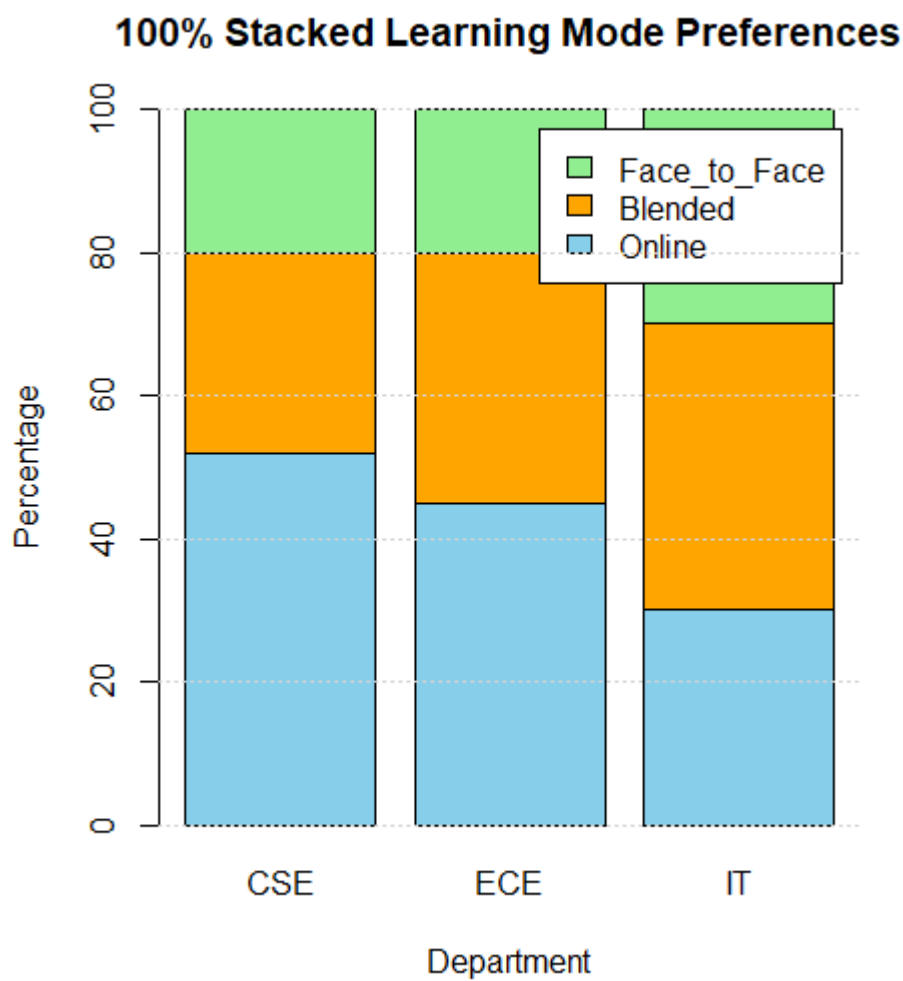
```

```

names.arg = departments,
col = c("skyblue", "orange", "lightgreen"),
main = "100% Stacked Learning Mode Preferences",
xlab = "Department",
ylab = "Percentage",
ylim = c(0, 100),
legend.text = TRUE
)

grid(nx = NA, ny = NULL)

```



Because each department totals 100%, the stacked bars have equal height.

3. Which chart is better?

Answer

The side-by-side bar chart is better for comparing individual proportions between departments because all bars start from a common zero baseline.

For example:

[
CSE=52%
]

[
ECE=45%
]

[
IT=30%
]

The differences are immediately visible.

A 100% stacked bar chart is better for showing the **composition of each department**, but comparing individual segments is more difficult.

4. Why are separate pie charts less effective?

Separate pie charts require comparison of angles and areas across different circles. This makes it difficult to accurately compare the same category between departments. Bar charts are easier because their lengths share a common baseline.

5. Misleading design

A truncated y-axis can exaggerate differences.

For example:

48% vs 52%

is only a 4 percentage-point difference.

But if the y-axis starts at 45%, the visual difference may appear much larger.

Final answer

A side-by-side bar chart is preferred for comparing individual proportions. A 100% stacked bar chart is useful for showing composition. Separate pie charts are less effective for cross-department comparisons.

Q4. Nested Proportions – Treemap and Sunburst

Given data

```
Category <- c(
  "Electronics", "Electronics", "Electronics",
  "Home", "Home", "Home",
  "Fashion", "Fashion", "Fashion"
)
```

```
Subcategory <- c(
  "Mobiles", "Laptops", "Accessories",
  "Furniture", "Appliances", "Decor",
  "Men", "Women", "Kids"
)
```

```
Revenue <- c(24, 18, 8, 14, 10, 6, 7, 8, 5)
```

```
revenue_data <- data.frame(
  Category,
  Subcategory,
  Revenue
)
```

The values are in ₹ lakh.

Important calculation

The supplied values total:

```
[  
24+18+8+14+10+6+7+8+5=100  
]
```

Therefore:

```
[  
Total=100\text{ lakh}=\text{₹}1\text{ crore}  
]
```

The question states ₹1.2 crore, but the supplied data totals only ₹1 crore.

Mention this discrepancy in the exam if necessary.

1. Treemap in R

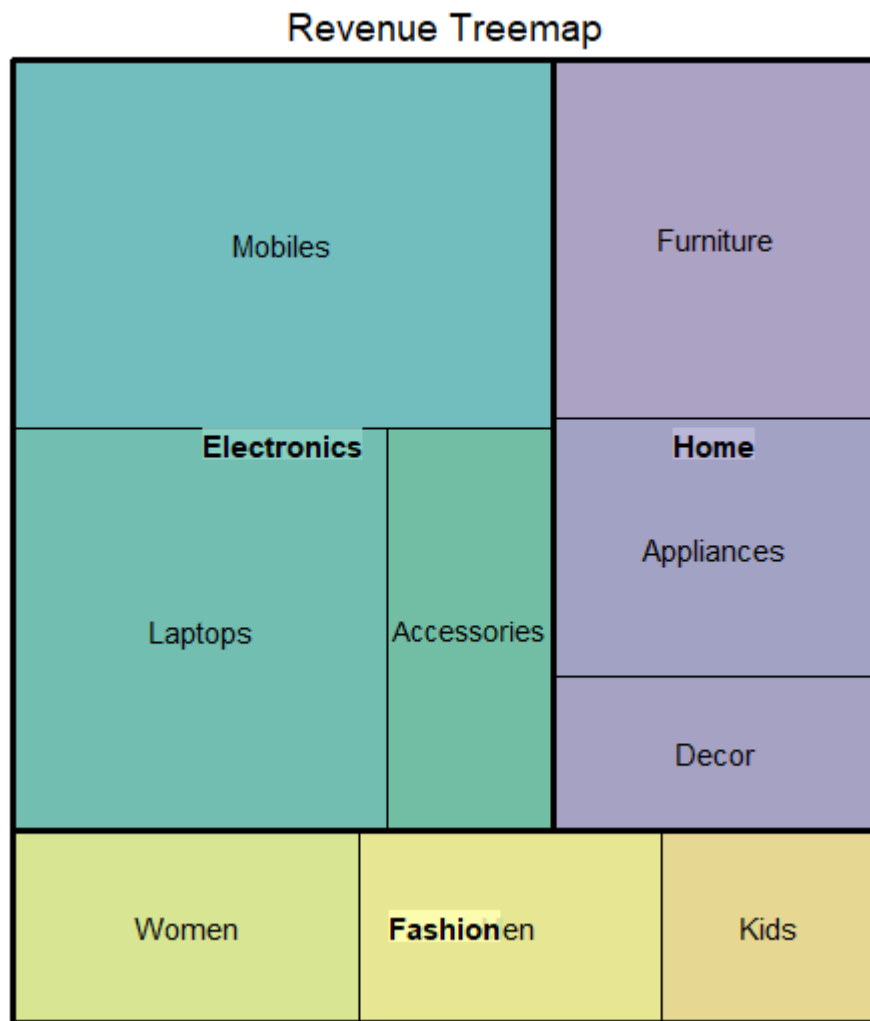
Install the package:

```
install.packages("treemap")
```

Then:

```
library(treemap)
```

```
treemap(  
  revenue_data,  
  index = c("Category", "Subcategory"),  
  vSize = "Revenue",  
  title = "Revenue Treemap",  
  palette = "Set3"  
)
```



The area of each rectangle represents revenue.

2. Sunburst Chart in R

Install:

```
install.packages("plotly")
```

Then:

```
library(plotly)
```

```
fig <- plot_ly(  
  revenue_data,  
  labels = ~Subcategory,
```

```
parents = ~Category,  
values = ~Revenue,  
type = "sunburst"  
)  
  
fig <- fig %>%  
  layout(  
    title = "Revenue Sunburst Chart"  
  )  
  
fig
```

3. Calculate category totals

```
category_total <- aggregate(  
  Revenue ~ Category,  
  data = revenue_data,  
  sum  
)
```

category_total

Expected result:

Category	Revenue
----------	---------

Electronics	50
-------------	----

Home	30
------	----

Fashion	20
---------	----

Largest category

Electronics:

$$[24+18+8=50\text{ lakh}]$$

Home:

$$[14+10+6=30\text{ lakh}]$$

Fashion:

$$[7+8+5=20\text{ lakh}]$$

Therefore:

$$[\boxed{\text{Electronics}}]$$

is the largest category.

Based on the supplied ₹100 lakh:

$$[\frac{50}{100}\times 100=50\%]$$

Therefore:

$$[\boxed{\text{Electronics}=50\%}]$$

If ₹120 lakh is used as the total

$$[\frac{50}{120}\times 100=41.67\%]$$

So write:

Based on the supplied subcategory data, Electronics contributes 50% of the total. However, if the stated total of ₹1.2 crore is used, Electronics contributes 41.67%. The supplied values themselves total only ₹1 crore.

4. Treemap vs Sunburst

Feature	Treemap	Sunburst
Representation	Rectangles	Concentric rings
Hierarchy	Yes	Yes
Magnitude comparison	Excellent	Good
Hierarchical relationship	Good	Excellent
Space efficiency	High	Moderate
Best use	Comparing revenue	Showing hierarchy

Final answer

A treemap is better when comparing the relative magnitude of categories because area directly represents revenue. A sunburst is better for displaying parent-child relationships and understanding hierarchical structure.

Area encoding limitation

Area encoding can become difficult to interpret for very small or irregularly shaped regions. Humans are generally better at comparing lengths than areas.

Q5. Flow Proportions – Sankey Diagram

Given data

```
flow_data <- data.frame(  
  Origin = c(  
    "Residential", "Residential", "Residential", "Residential",  
    "Suburban", "Suburban", "Suburban", "Suburban",  
    "Rural", "Rural", "Rural", "Rural"  
  ),
```

```
  Mode = c(  
    "Bus", "Bus", "Train", "Train",
```

```
"Bus", "Bus", "Train", "Train",  
"Bus", "Bus", "Train", "Train"  
)
```

```
Destination = c(  
"Central", "Industrial", "Central", "Industrial",  
"Central", "Industrial", "Central", "Industrial",  
"Central", "Industrial", "Central", "Industrial"  
)
```

```
Commuters = c(  
35, 15, 25, 10,  
20, 10, 30, 15,  
12, 8, 10, 10  
)  
)
```

Total:

```
[  
200  
]
```

1. Sankey Diagram in R

Install Plotly:

```
install.packages("plotly")
```

Then:

```
library(plotly)
```

```
labels <- c(  
"Residential",
```

```
"Suburban",  
"Rural",  
"Bus",  
"Train",  
"Central",  
"Industrial"  
)
```

We need to convert the categories into node numbers:

```
flow_data$Origin_ID <- match(flow_data$Origin, labels) - 1
```

```
flow_data$Mode_ID <- match(flow_data$Mode, labels) - 1
```

```
flow_data$Destination_ID <- match(flow_data$Destination, labels) - 1
```

Create the Sankey:

```
fig <- plot_ly(  
  type = "sankey",  
  
  node = list(  
    label = labels,  
    pad = 20,  
    thickness = 20  
  ),
```

```
  link = list(  
    source = c(  
      flow_data$Origin_ID,  
      flow_data$Mode_ID  
    ),  
    target = c(  
      flow_data$Mode_ID,  
      flow_data$Destination_ID  
    )  
  )  
)
```

```

    flow_data$Mode_ID,
    flow_data$Destination_ID
  ),

  value = c(
    flow_data$Commuters,
    flow_data$Commuters
  )
)

fig <- fig %>%
  layout(
    title = "Commuter Flow: Origin → Mode → Destination"
  )

fig

```

2. Parallel Sets in R

Plotly can also create a parallel-categories diagram.

```

library(plotly)

fig <- plot_ly(
  flow_data,
  type = "parcats",
  dimensions = list(
    list(
      label = "Origin",

```

```

    values = flow_data$Origin
  ),
  list(
    label = "Mode",
    values = flow_data$Mode
  ),
  list(
    label = "Destination",
    values = flow_data$Destination
  )
)

fig <- fig %>%
  layout(
    title = "Parallel Sets of Commuter Flows"
  )

fig

```

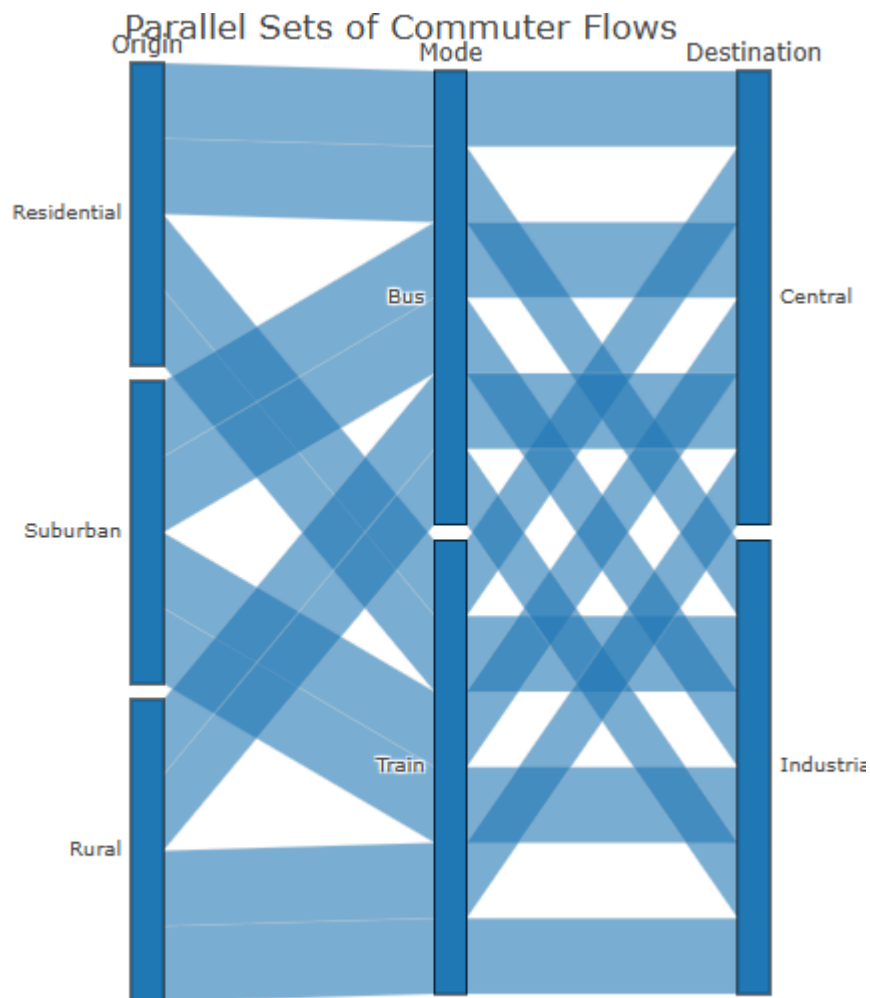
3. Three Largest Flows

We can sort the data:

```

flow_sorted <- flow_data[
  order(-flow_data$Commuters),
]

```



flow_sorted

The three largest are:

1. Residential → Bus → Central

[
35
]

Percentage:

[
 $\frac{35}{200} \times 100 = 17.5\%$
]

2. Suburban → Train → Central

```
[
30
]
```

Percentage:

```
[
\\frac{30}{200}\\times100=15%
]
```

3. Residential → Train → Central

```
[
25
]
```

Percentage:

```
[
\\frac{25}{200}\\times100=12.5%
]
```

R code for percentages

```
flow_data$Percentage <-
```

```
  flow_data$Commuters / sum(flow_data$Commuters) * 100
```

```
flow_data[
  order(-flow_data$Commuters),
  c("Origin", "Mode", "Destination",
    "Commuters", "Percentage")
][1:3, ]
```

Result

Rank Flow		Commuters Percentage
1	Residential → Bus → Central	35 17.5%
2	Suburban → Train → Central	30 15%
3	Residential → Train → Central	25 12.5%

Combined:

[
17.5+15+12.5=45%
]

Therefore:

[
 $\boxed{45\%}$
]

of commuters are represented by the three largest flows.

4. Sankey vs Parallel Sets

Feature	Sankey	Parallel Sets
Flow magnitude	Excellent	Good
Flow direction	Excellent	Good
Categorical relationships	Good	Excellent
Dominant flows	Excellent	Good
Multiple categories	Good	Excellent
Best use	Flow analysis	Category relationships

Final answer

A Sankey diagram is better for communicating flow magnitude and identifying dominant pathways because the width of each link represents the number of commuters. Parallel sets are useful for exploring relationships between multiple categorical variables.

5. How poor design can mislead

The link width must be proportional to the flow.

For example:

[
30\text{ commuters}
]

should have approximately **three times the visual width** of:

[
10\text{ commuters}
]

If this relationship is not maintained, the visualization can exaggerate or minimize the importance of a flow.

Poor category ordering can also create too many crossing links, making the diagram difficult to understand.

Final answer

Incorrect link-width scaling can misrepresent flow magnitude. Poor ordering of categories can create excessive crossings and make the flow difficult to follow. Therefore, Sankey and parallel-set diagrams should use proportional widths and logical category ordering.